

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет программной инженерии и компьютерных технологий

Тестирование программного обеспечения

Лабораторная работа № 2

Выполнили студенты

Набокова Алиса,

Шнейдерис Герардас

Группа № Р3320

Преподаватель:

Демурчян Владимир Варламович

г. Санкт-Петербург

2026

Лабораторная работа № 2	1
Задание:	3
Задание 1:	6
С помощью анализа эквивалентности логарифмических функции были выделены следующие (общие для них всех) участки и точки:	10
Вывод:	11

Вариант: 9471

Задание:

Провести интеграционное тестирование программы, осуществляющей вычисление системы функций (в соответствии с вариантом).

$$\begin{cases} \left(\left(\left(\left(\sec(x)^2 + \sin(x) \right)^2 - \tan(x) \right) - (\cos(x)^3) \right) & \text{if } x \leq 0 \\ \left(\frac{\left(\frac{(\log_{10}(x))^3 \cdot \log_5(x)}{\ln(x)} \right)^3}{(\log_{10}(x) + \ln(x)) + \log_{10}(x)} \right) & \text{if } x > 0 \end{cases}$$

Правила выполнения работы:

1. Все составляющие систему функции (как тригонометрические, так и логарифмические) должны быть выражены через базовые (тригонометрическая зависит от варианта; логарифмическая - натуральный логарифм).

2. Структура приложения, тестируемого в рамках лабораторной работы, должна выглядеть следующим образом (пример приведён для базовой тригонометрической функции $\sin(x)$):



3. Обе "базовые" функции (в примере выше - $\sin(x)$ и $\ln(x)$) должны быть реализованы при помощи разложения в ряд с задаваемой погрешностью. Использовать тригонометрические / логарифмические преобразования для упрощения функций ЗАПРЕЩЕНО.

4. Для КАЖДОГО модуля должны быть реализованы табличные заглушки. При этом, необходимо найти область допустимых значений функций, и, при необходимости, определить взаимозависимые точки в модулях.

5. Разработанное приложение должно позволять выводить значения, выдаваемое любым модулем системы, в csv файл вида «X, Результаты модуля (X)», позволяющее произвольно менять шаг наращивания X. Разделитель в файле csv можно использовать произвольный.

Порядок выполнения работы:

1. Разработать приложение, руководствуясь приведенными выше правилами.
2. С помощью JUNIT5 разработать тестовое покрытие системы функций, проведя анализ эквивалентности и учитывая особенности системы функций. Для анализа особенностей системы функций и составляющих ее частей можно использовать сайт <https://www.wolframalpha.com/>.
3. Собрать приложение, состоящее из заглушек. Провести интеграцию приложения по 1 модулю, с обоснованием стратегии интеграции, проведением интеграционных тестов и контролем тестового покрытия системы функций.

Выполнение:

Выбирая тестовое покрытие для тригонометрических функций, воспользуемся анализом эквивалентности

Для каждой функции тестируем точки, где функция имеет экстремумы, а также изменение знака. Выделенные области показывают, где функция “по мнению тестировщика” ведет себя схожим образом. То есть области отличаются скоростью возрастания и убывания

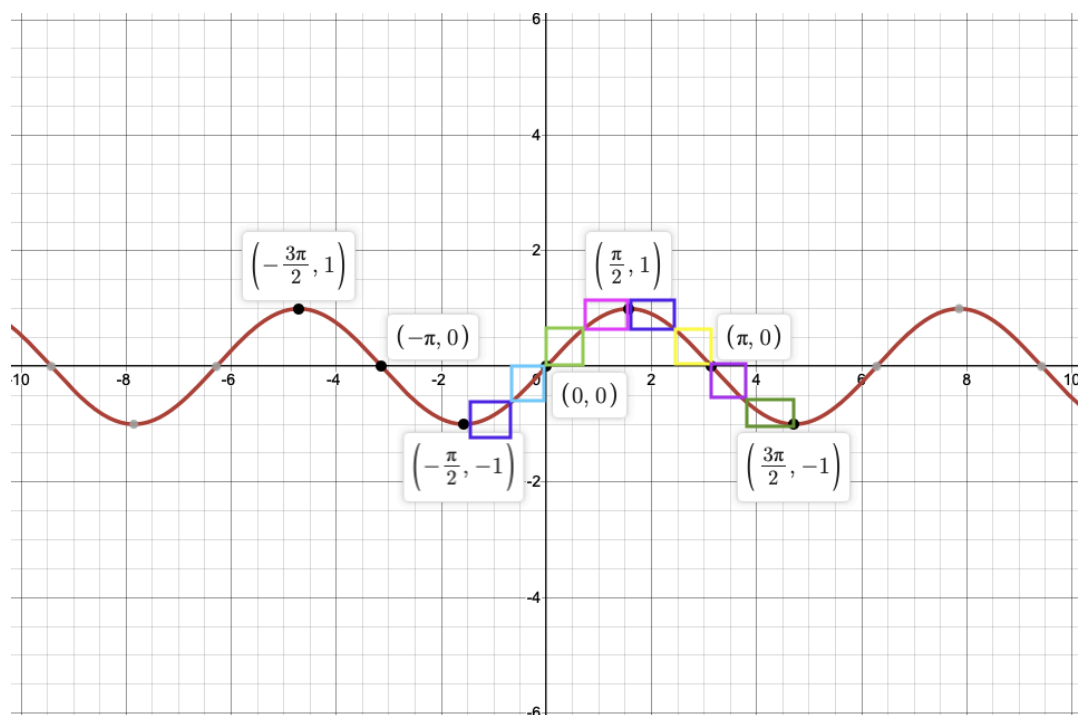


Рисунок 1 - Анализ эквивалентности функции $\sin(x)$

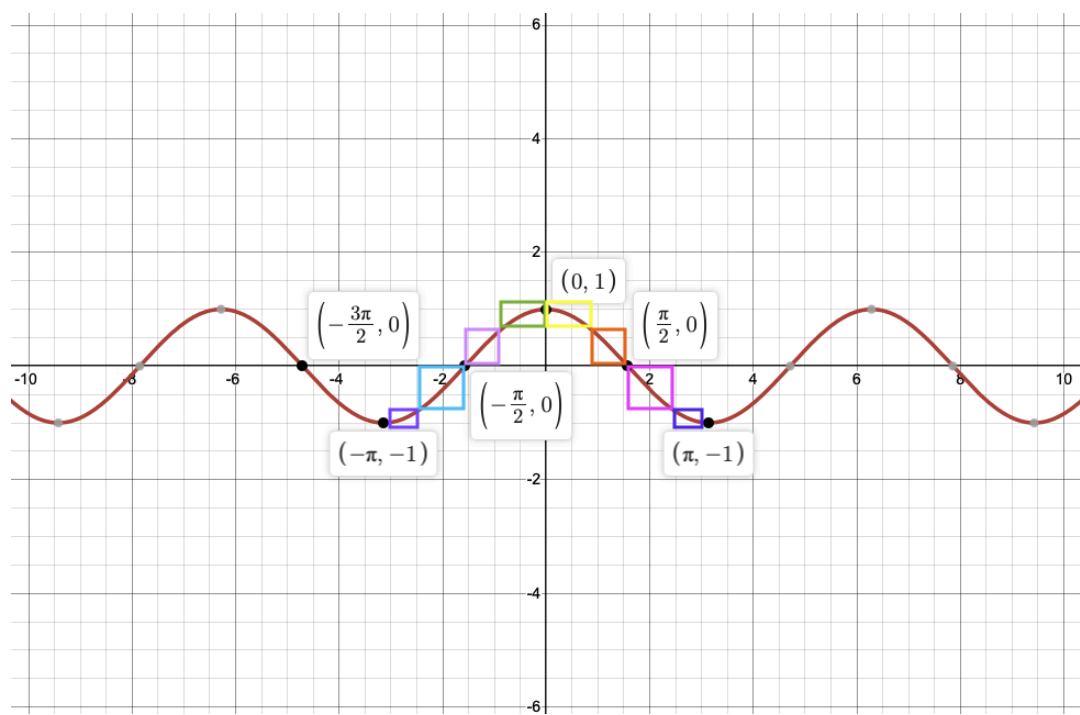


Рисунок 2 - Анализ эквивалентности функции $\cos(x)$

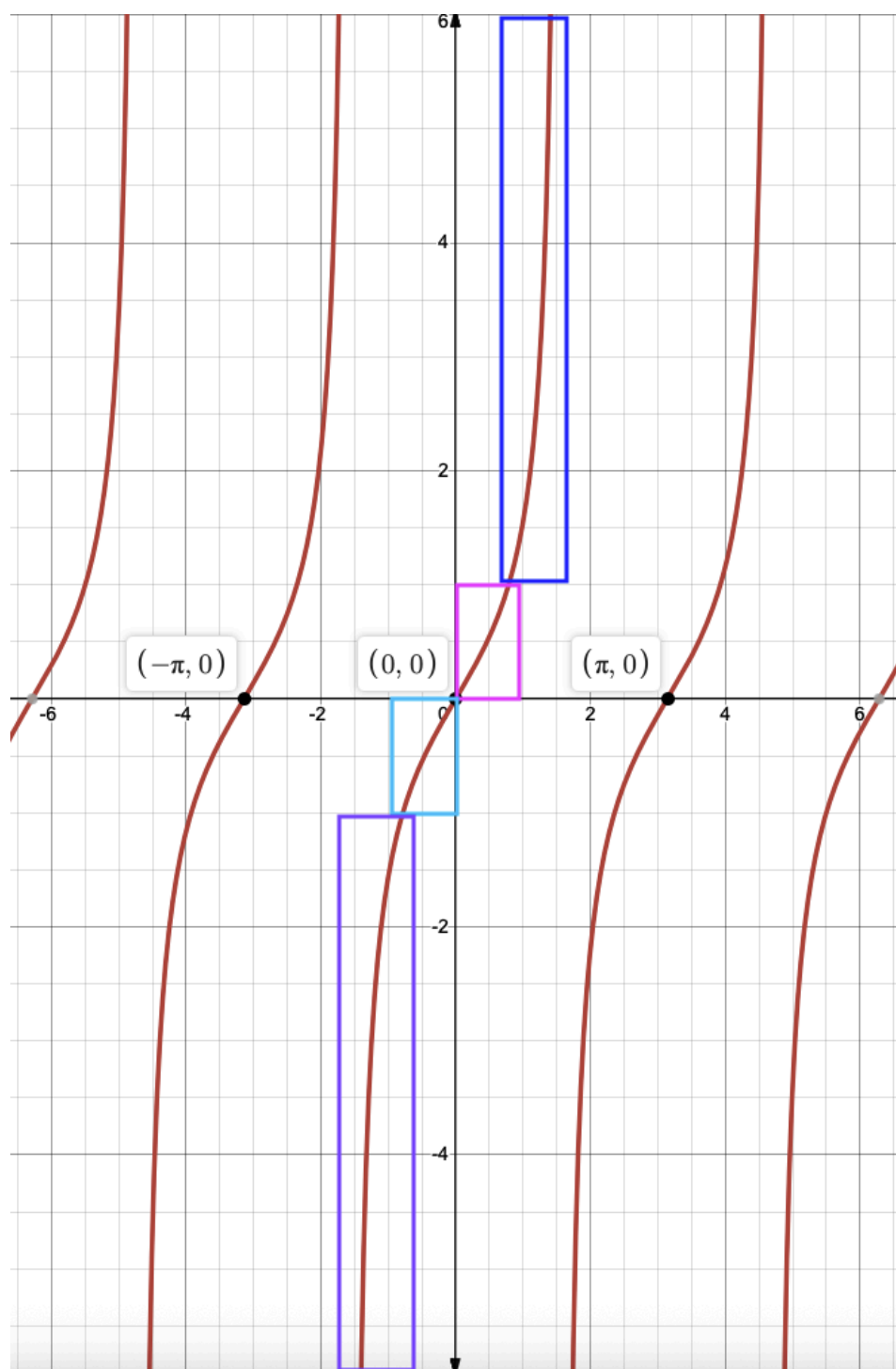


Рисунок 3 - Анализ эквивалентности функции $\tan(x)$

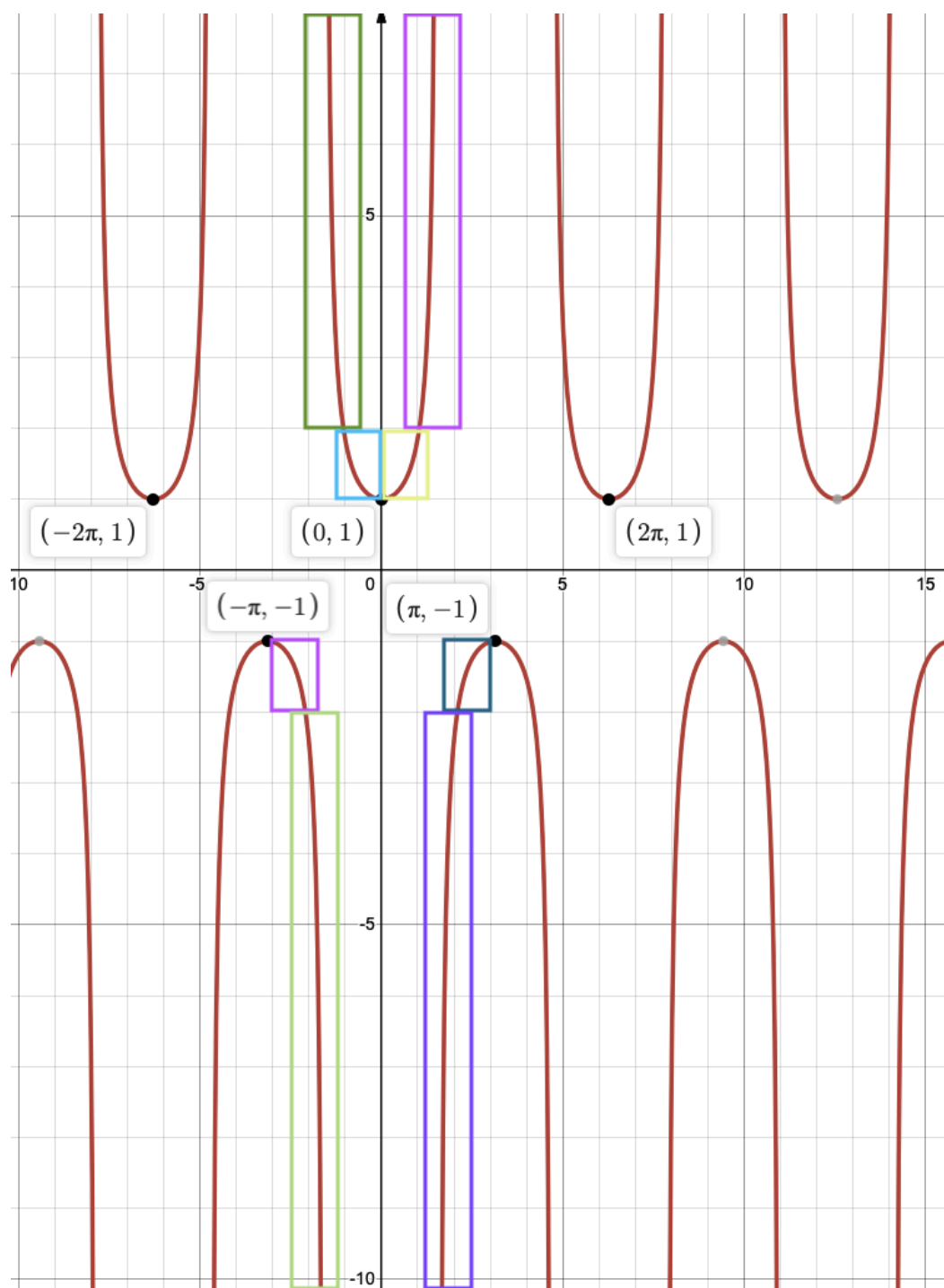


Рисунок 4 - Анализ эквивалентности функции $\sec(x)$

С помощью анализа эквивалентности логарифмических функции были выделены следующие (общие для них всех) участки и точки:

- $[-\infty; 0]$ - на этом промежутке функция неопределенна
- $(0; \varepsilon]$ - большие по модулю отрицательные значения функции
- $[1 - \varepsilon; 1 + \varepsilon]$ - монотонное возрастание функции
- 1 - точка перемены знака
- большие положительные значения тоже должны быть протестированы

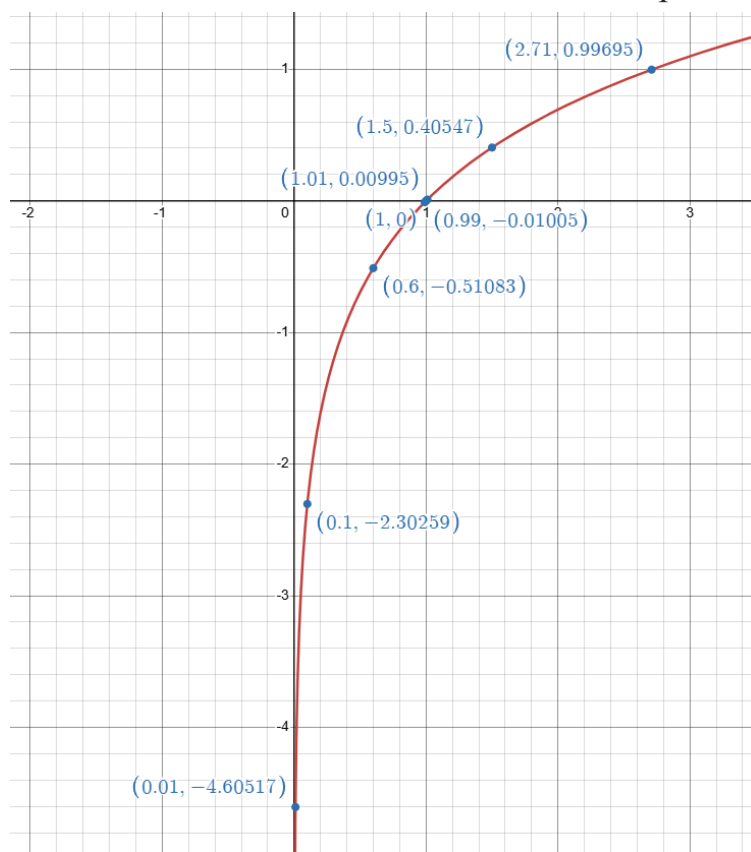


Рисунок 5 - тестируемые точки для натурального логарифма

NB: Для логарифмов с основанием также нужно протестировать некорректные основания: отрицательные и единицу

Затем мы провели интеграционное тестирование с помощью Mockito выбрав стратегию интеграцию снизу-вверх. Код тестов и реализацией доступен по [ССЫЛКЕ](#)

Element ^	Class, %	Method, %	Line, %	Branch, %
org.example	75% (9/12)	73% (17/23)	75% (54/72)	53% (15/28)
log	100% (3/3)	83% (5/6)	90% (20/22)	58% (7/12)
trig	71% (5/7)	69% (9/13)	66% (30/45)	42% (6/14)
MainKt	0% (0/1)	0% (0/1)	0% (0/1)	100% (0/0)
System	100% (1/1)	100% (3/3)	100% (4/4)	100% (2/2)

Рисунок 6 - полученные тестовое покрытие

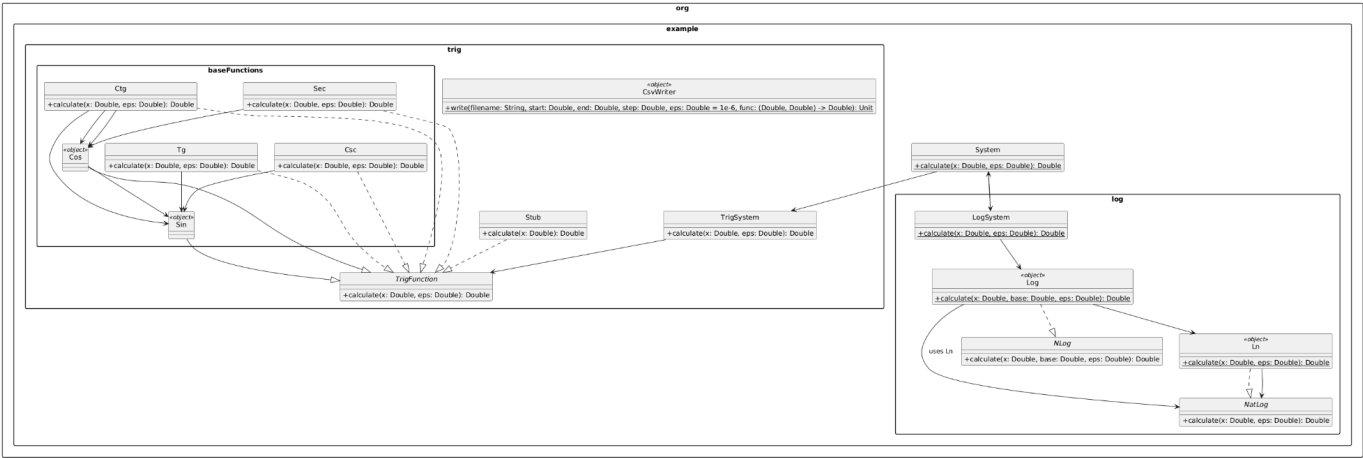


Рисунок 7 - UML диаграмма разработанного приложения

Вывод:

мы познакомились с интеграционным тестированием, BDD, Mockito и написали на котлине.

Me looking at Java code
after learning Kotlin

